DIALITE: Discover, Align and Integrate Open Data Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We demonstrate a novel table discovery pipeline called DIALITE that allows users to discover, integrate and analyze open data tables. DIALITE has three main stages. First, it allows users to discover tables from open data platforms using state-of-the-art table discovery techniques. Second, DIALITE integrates the discovered tables to produce an integrated table. Finally, it allows users to analyze the integration result by applying different downstreaming tasks over it. Our pipeline is flexible such that the user can easily add and compare additional discovery and integration algorithms.

CCS CONCEPTS

• **Information systems → Information integration**.

KEYWORDS

Data Lakes, Data Discovery, Data Integration, Full Disjunction

ACM Reference Format:
Aamod Khatiwada, Roee Shraga, and Renée J. Miller. 2023. DIALITE: Discover, Align and Integrate Open Data Tables. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, Seattle, WA, USA, 4 pages. https://doi.org/10.1145/3555041.3589732

1 INTRODUCTION

Data discovery has become an important component in data science pipeline. The discovery process uses techniques such as keyword search [13] and table search [4, 7, 9] to discover a set of tables (datasets). Data scientists use such tables to support decision-making processes, train machine learning models, perform statistical analysis, and so on. After discovery, a natural step is to integrate the discovered tables. An integrated table provides a unified view of the data and allows users to run queries and analyses that go beyond a single table.

While integrating tables, we intend to combine tuples from different tables in a maximal way such that the integrated tuples carry as much information as possible. This enriches the analysis and decision-making process after integration and also improves the quality of downstream applications. The widely known outer-join [12] operator is not associative and does not aim to maximize the connections among the integrated tuples [6]. Accordingly, *Full Disjunction (FD)* [6] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [11]. FD can be viewed as an associative version of outer join [2] and has been used to integrate information across relational tables [2, 10] and web tables [10]. In a recent paper [8], we proposed ALITE, a new algorithm based on Full Disjunction to integrate tables discovered in data lakes. ALITE was shown to be correct and faster than the existing FD algorithms [2, 10] while integrating real open data lake tables in practice [8]. Additional details on other integration operators and their comparison against FD are available in the full ALITE paper [8] where FD was shown to be a better semantics for integration because it produces a result over which a downstream task like entity-resolution performs more accurately.

In this demonstration, we propose DIALITE, a novel system that discovers, aligns, and integrates open data tables. It extends the aforementioned ALITE [8], which does not perform discovery but instead takes a set of tables as input, aligns the matching columns using holistic schema matching, and applies the FD to get an integrated table. DIALITE offers state-of-the-art systems for different table discovery tasks [7, 14] before applying ALITE to integrate them. Furthermore, DIALITE also offers new downstream analytics (that have not been previously considered) to evaluate the quality of integration. Specifically, DIALITE allows users to

(1) Upload a table or randomly generate one using GPT-3 as a query and discover related tables from a given data lake that are unionable [7] or joinable [14] with the query table. Apart from the available table discovery algorithms, DIALITE also allows users to add new algorithms for table search based on their preference.
(2) Integrate the discovered tables (or upload a set of tables to be integrated) using a novel table integration system called ALITE [8]. Besides ALITE, we allow users to add new integration operators within our extendible architecture.
(3) Analyze and compare the table integrated using ALITE against alternative integration techniques by performing downstream applications. In our demo, we consider outer join as an alternative integration technique (or other queries/methods added by the user) and present data analytics (common aggregations and statistics) and entity-resolution as downstream applications. Both, when applied over real tables (which can be incomplete) will show the dramatic difference between maximally integrating information using FD vs. using outer joins.

Related Work. To the best of our knowledge, DIALITE is the first system that enables a full table search pipeline starting from

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589732

table discovery, followed by table integration and downstream applications over the integrated table. An earlier system called COCOA [3] focuses on finding joinable tables with correlated attributes that expand the query table. After finding the tables, COCOA applies LEFT JOIN between the query table and the discovered table as the integration operator. DIALITE, on the other hand, finds a set of related tables that can be integrated with the query table (including unionable and joinable tables). Auctus [1] also discovers related tables and integrates table pairs by applying inner join or union. Unlike Auctus, DIALITE does not limit the number of tables to be integrated. In addition, DIALITE uses FD to integrate the tables, which has been shown in theory to maximize the connections among the tuples in the tables [11].

Existing table discovery techniques search for joinable, unionable, or related tables. **JOSIE** [14] and **LSH Ensemble** [15], for instance, take a query table as input with a query column marked by the user and returns a set of top-k tables that are joinable with the query table. **SANTOS** [7] takes a query table and discovers a set of top-k semantically unionable tables as output. Other table search techniques [4, 9] also focus on table discovery from large repositories. But the important question of how to integrate the discovered tables is not addressed. **ALITE** [8] provides a solution to this question. As the data lake tables may lack consistent and meaningful column headers, ALITE applies holistic schema matching over the set of searched tables and assigns a dummy column header called an Integration ID to the set of matching columns. Then, it applies a natural FD over the integration IDs to integrate the tables. The integration process outputs an integrated table with maximally integrated tuples [6, 11].

2 SYSTEM DESCRIPTION

DIALITE, outlined in Fig. 1, has three stages (discover, align & integrate, and analyze) which are included in our demo plan (Sec. 3).

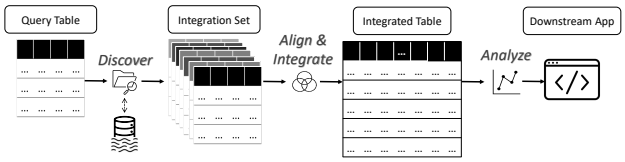


Figure 1: An overview of DIALITE.

DIALITE offers various options for discovery, alignment, integration, and analysis, including the ability for users to implement their own methods. The pipeline begins with a user-provided query table Q. However, we also allow the user to randomly generate a table using GPT-3 [5]. We now detail the components.

2.1 Discover

Given a query table Q, DIALITE uses a data discovery method to find tables in a data lake (table repository) $\mathcal{D}$ that are unionable [7], joinable [14], or simply semantically similar. DIALITE allows the user to choose among existing discovery algorithms including SANTOS [7] and LSH Ensemble [15]. Alternatively, a user can use their own discovery algorithm by implementing a similarity method between two tables. As we apply ALITE [8] for alignment and integration, the discovery phase is agnostic to the type of search. ALITE, aiming to maximize the connections among the tables, decides how the tables should be integrated. The output of discovery is an *integration set* $D \subseteq \mathcal{D}$, a set of tables to be integrated, including the query table. Our discovery techniques allow users to control the number of tables returned, and our system permits users to select a subset of the discovered tables to be integrated.

2.2 Align and Integrate

Given an integration set D, DIALITE uses ALITE [8] to integrate the tables. The integration set can be derived from the discovery part (Sec. 2.1) or provided as input by the user. The latter represents a traditional data integration scenario where the integration set is given. As shown in Fig. 1, this stage outputs an integrated table.

ALITE is composed of two main parts, namely align and integrate. Note that we do not require reliable column headers in the integration set. The first part of ALITE (Align) applies holistic schema matching to identify common columns in the integration set. The matched columns are given the same integration ID. Using these ids as attribute names, the integrate part applies (natural) FD. Specifically, ALITE uses a new holistic schema matching algorithm that was shown to outperform state-of-the-art matchers. ALITE uses a novel algorithm to compute the FD shown to be correct and efficient on real tables (including tables having nulls) [8]. DIALITE also allows users to add alternative integration operators, e.g., outer join, which is included for demonstration.

2.3 Analyze

Given an integrated table (that can also be uploaded as a file by the user), DIALITE allows the user to explore the benefits of integration. Specifically, the user can choose a downstream application to apply over the integrated table. A simple application is an aggregation query that can be applied over the integrated table as we illustrate in Sec. 3.1. In Sec. 3.2 we also present a more complex downstream application, entity-resolution (ER).

2.4 Implementation

DIALITE is implemented in Python 3.8 and the demonstration uses a web application.[1] Within our pipeline, we use SANTOS [2], LSH Ensemble [3] and ALITE [4] using their publicly available code. Also, we use the *py_entitymatching* package to show ER as a downstream application.[5] We allow users to interact with the system after each step so that they can validate the intermediate results.

3 DEMONSTRATION PLAN

Next, we illustrate our demonstration. The link to a demonstration video is available in the github repository.[1] Our demo contains two parts. First, we present the use case of the pipeline as described in Sec. 2. This part is actually composed of three demonstration items, that can be demonstrated independently. Then, we demonstrate DIALITE's extensibility to new algorithms for discovery, integration, and analysis.

[1] https://github.com/northeastern-datalab/dialite
[2] https://github.com/northeastern-datalab/santos
[3] https://github.com/ekzhu/datasketch
[4] https://github.com/northeastern-datalab/alite
[5] https://github.com/anhaidgroup/py_entitymatching

3.1 DIALITE Use case

Discovery. The users of DIALITE would be able to upload a query table in CSV format from an existing pool of tables.[6] Note that we will provide a data lake for the users to use in the demonstration. The tables in this data lake are real tables from open data and are currently preprocessed for our discovery algorithms [7, 15]. Specifically, the indexes used in SANTOS [7] and LSH Ensemble [15] are built offline, i.e., they are already available for the user to use. The users can easily preprocess and link their own data lake.

The set of discovered tables by all the table discovery systems is stored for the next step. As there may be an overlap in unionable and joinable search results, we persist *the set of tables* found by all techniques to form an integration set.

T_1 (Query Table)

TID	Country	City	Vaccination Rate (1+ dose)
t_1	Germany	Berlin	63%
t_2	England	Manchester	78%
t_3	Spain	Barcelona	82%

T_2 (Retrieved unionable table)

TID	Country	City	Vaccination Rate (1+ dose)
t_4	Canada	Toronto	83%
t_5	Mexico	Mexico City	±
t_6	USA	Boston	62%

T_3 (Retrieved joinable table)

TID	City	Total Cases	Death Rate (per 100k residents)
t_7	Berlin	1.4M	147
t_8	Barcelona	2.68M	275
t_9	Boston	263k	335
t_{10}	New Delhi	2M	158

Figure 2: Tables detailing COVID-19 cases in different places. The symbol ± represents null values present in the input tables ("*missing nulls*").

Example 1. *Consider tables T_1, T_2 and T_3 about COVID-19 cases in different places shown in Fig. 2. Here, TID (Tuple ID) is not a real data column and it is added only to refer to the tuples. Let, T_1 be the query table, and tables T_2 and T_3 reside in the data lake repository. In our demo, a user selects* `City` *as an intent column and query column to search for a unionable table using SANTOS [7] and a joinable table using LSH Ensemble [15] respectively. Please see respective papers for additional details on the search algorithms. Let, T_2 and T_3 be results of the unionable and joinable searches respectively. So the result of the discovery step is an integration set of tables T_1, T_2, and T_3. Note that the names of columns are presented for simplicity and are not used by the discovery techinques, which are designed for the ambiguty of data lakes, i.e., unreliable/missing metadata.*

Align and Integrate. Once the integration set is formed, DIALITE allows user to apply ALITE's holistic schema matching to generate integration IDs. Over such IDs, we apply ALITE's FD.

Example 2. *Consider the integration set of tables T_1, T_2 and T_3 shown in Fig. 2 is formed after table discovery step as illustrated in Ex. 1. DIALITE applies ALITE's integration algorithm over these tables that returns an integrated table as shown in Fig. 3. The integration semantics is explained in ALITE paper [8].*

[6]For the demonstration itself, we also allow users to randomly generate a query table (see Sec. 3.2). Also, a user may choose to upload their query table noting that it may be off-topic wrt the data lake, which may yield no results.

FD(T_1, T_2, T_3) (Integrated Table by ALITE)

OID	TIDs	Country	City	Vaccination Rate (1+ dose)	Total Cases	Death Rate (per 100k residents)
f_1	{t_1, t_7}	Germany	Berlin	63%	1.4M	147
f_2	{t_2}	England	Manchester	78%	⊥	⊥
f_3	{t_3, t_8}	Spain	Barcelona	82%	2.68M	275
f_4	{t_4}	Canada	Toronto	83%	⊥	⊥
f_5	{t_5}	Mexico	Mexico City	±	⊥	⊥
f_6	{t_6, t_9}	USA	Boston	62%	263k	335
f_7	{t_{10}}	⊥	New Delhi	⊥	2M	158

Figure 3: Result of applying ALITE over the tables in Fig. 2. The symbol ⊥ represents the null values produced during the integration due to missing information ("*produced nulls*").

Analyze. After integration, the next feature that ALITE offers is the analysis of the integrated table. We show the use of an aggregation query over the integrated table.

Example 3. *With the integrated table, we now allow the user to use queries that go beyond the single tables. For example, over the integrated table (Fig. 3), the user can find that Boston is the city with the lowest vaccination rate and Toronto has the highest. Trying to understand the reason for that, the user may explore the relationship between vaccination rates (given in T_1 and T_2), number of cases and death rates (given in T_3). For example, the user can compute the correlation between vaccination and death rates that shows a positive (pearson) correlation of 0.16 and (somewhat surprising) correlation of 0.9 between case numbers and vaccination rates. While a bit counter-intuitive, the analysis reveals an interesting insight about the nature of vaccinations, suggesting that in cities with higher death rates and more cases, the government is focusing on vaccination programs and the people are more willing to vaccinate.*

3.2 DIALITE Extendibility

As described in Sec. 2, the user can extend DIALITE by implementing their own alternative components in the pipeline. Specifically, we aim to demonstrate the ability of users to implement (using python code) new discovery algorithms, integration methods and perform their required analysis.

Example 4. *For illustration, a sample code snippet to add a new joinable table discovery algorithm is provided in Fig. 4. The user basically implements a similarity function between two datasets (df1 and df2) that is used by DIALITE for table discovery.*

```
def new_joinability_discovery_algorithm(df1, df2):
    join_df = pd.merge(df1, df2, how ='inner')
    return len(join_df)/max(len(df1), len(df2))
```

Figure 4: Implementing user-defined discovery algorithm based on inner join.

We also consider a scenario where the user may not have a query table to start the analysis. So, DIALITE allows users to use simple prompts to generate the query table for the analysis. We demonstrate this feature using GPT-3 based implementation [5] and allow the user to generate a query table based on prompt as

Integrating Data Lake Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Wolfgang Gatterbauer
Northeastern University
Boston, Massachusetts, USA
w.gatterbauer@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We have made tremendous strides in providing tools for data scientists to discover new tables useful for their analyses. But despite these advances, the proper integration of discovered tables has been under-explored. An interesting semantics for integration, called Full Disjunction, was proposed in the 1980's, but there has been little progress in using it for data science to integrate tables culled from data lakes. We provide ALITE, the first proposal for scalable integration of tables that may have been discovered using join, union or related table search. We empirically show that ALITE can outperform previous algorithms for computing the Full Disjunction. ALITE relaxes previous assumptions that tables share common attribute names (which completely determine the join columns), are complete (without null values), and have acyclic join patterns. To evaluate ALITE, we develop and share three new benchmarks for integration that use real data lake tables.

PVLDB Reference Format:
Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. Integrating Data Lake Tables. PVLDB, 16(4): 932 - 945, 2022.
doi:10.14778/3574245.3574274

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/northeastern-datalab/alite.

1 INTRODUCTION

The number of public datasets has grown immensely in open data platforms [56, 57, 78]. Also, individual corporations have a wealth of data stored in their own data lakes. Analyzing and integrating such datasets can help governments and enterprises in making decisions and plans. Data scientists, as the main users of data, use different techniques to discover datasets such as keyword search [11, 12, 58, 75] and table search (using the data within their table as a query) [10, 28, 48, 50, 57, 80]. Such a process usually outputs a collection of data lake tables that may enrich their analysis [56]. Existing techniques usually discover unionable [12, 41, 48, 57], joinable [26, 55, 75, 77, 78], and related tables [10, 19, 77].

Example 1. *Consider the data lake tables about football stadiums shown in Fig. 1. We have added a* `TID` *(Tuple ID) column in each table to permit us to refer to tuples. Assume that a data scientist uses table* T_1 *as a* query table *to search for the top-2 unionable tables [57] and the top-2 joinable tables [78] from a data lake. Let* T_2 *and* T_3 *be the union search results and* T_4 *and* T_5 *be the join search results. Join search finds tables that join on an indicated column (in this case* `Location`*), but does not discover if there are other common (integratable) columns. For simplicity, assume that the common columns on these tables are already detected and have identical column headers. Note that in practice this will not be the case.*

After discovery, data scientists would often integrate the discovered tables before analyzing and applying statistical tools. Such integration not only extends their data but also allows them to answer queries that go beyond a single table. Consider Tables T_2, T_3 and T_4 in Fig. 1 and assume a football team has data scientists assisting in finding a new coach. Specifically, the team looks for an experienced coach who has handled teams playing in front of large crowds in new stadiums. So, they may use queries such as *"coaches who coach teams having stadiums established after 2000, that accommodate at least 50 thousand spectators"*. The information required here goes beyond a single table. In our example, one needs (at least) to integrate T_2, T_3 and T_4 to obtain such facts, for example, `Dan Campbell` who coaches the `Detroit Lions` that uses `Ford Field` Stadium established in `2002` and having a capacity of `65k` (f_7 in Fig. 2). Prior search methods do not address this "post-discovery" phase and do not answer the important question of how to integrate tables (relations) obtained by table search technique(s).

Example 2. *The standard relational union operator needs all tables to have exactly the same schema. However, this is not the case even for union search results (where tables that union on a subset of attributes can be retrieved) [57]. So to integrate the tables in Fig. 1, one can project out non-common columns and union on only the common columns. For* T_1, T_2, *and* T_3, *this would just leave* `Location`*. For the joinable tables, a join on only* `Location` *of* T_1 *with* T_4 *leads to tuples like* t_{11} *being omitted and the result has two* `Stadium` *attributes. Worse, the natural join operator, i.e.* $T_1 \bowtie T_4 \bowtie T_5$*, returns an empty set because* T_4 *and* T_5 *do not have joining tuples. The problem gets more complicated if we try to integrate all five tables using these operators.*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574274

T_1

TID	Stadium	Location	Team
t_1	NRG Stadium	Texas	Houston Texans
t_2	AT&T Stadium	Texas	Dallas Cowboys
t_3	Paul Brown	Ohio	±
t_4	Sofi Stadium	California	Angeles Chargers

T_2

TID	Stadium	Location	Opened
t_5	Soldier Field	Chicago	1924
t_6	Ford Field	Michigan	2002

T_3

TID	Team	Location	Coach
t_7	Houston Texans	Texas	Lovie Smith
t_8	Green Bay Packers	Wisconsin	Matt LaFleur
t_9	Detroit Lions	Michigan	Dan Campbell

T_4

TID	Stadium	Location	Capacity
t_{10}	NRG Stadium	Texas	±
t_{11}	Ford Field	Michigan	65k

T_5

TID	Stadium	Location	Team
t_{12}	Lambeau Field	Wisconsin	Green Bay Packers
t_{13}	±	Ohio	Cleveland
t_{14}	Sofi Stadium	California	±

Figure 1: Tables about football stadiums, their locations and home teams. The objective is to integrate the five tables. TID is not a real data column and metadata like column headers may not be available in real data lake tables, but are used for illustration purposes. The symbol ± represents null values present in the input tables ("*missing nulls*").

Within the data integration literature, *Full Disjunction (FD)* [32] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [65]. Indeed, Rajamaran and Ullman describe FD as a relation with nulls (denoted by ⊥) such that every set of join-consistent tuples appears within an FD tuple, with a concrete value or ⊥ in each attribute not found within the set of tuples [65]. Here, join-consistent is defined as common attributes (attributes with the same name), so this is effectively a *natural FD*. The widely known outer-join [46, 66] is not associative (hence, the result may depend on the order in which tables are integrated) and does not aim to maximize the connections among the integrated tuples [32, 52].

Example 3. *Outer-join and outer-union keep all tuples and columns and pad non-matching tuples (respectively, columns) with nulls [16, 46]. The outer-union of the tables from Fig. 1 is depicted in Fig. 2(a). It does not maximally connect the facts in the original tables. Here, ± indicates a missing value (missing null) in the original tables and ⊥ represents a null introduced by the outer-union operator (produced null). In particular, outer union includes partial facts like t_{10} that are made redundant by more complete facts like t_1. Similar observations can be made of the outer-join results. So, Galindo-Legario defined the Full Disjunction (FD) [32]. Informally, it removes redundant facts and produces, in this case, the first 8 tuples (mustard colored) of Fig. 2(b). FD can be viewed as an associative version of outer join [17] and has been used to integrate information across relational tables [17, 59] and web tables [59]. Notice in this example, FD uses information from all five tables so it is important to be able to compute FD over (possibly large) sets of tables.*

But using FD in data lakes poses several important challenges: 1) We cannot rely on common attributes having the same name as in our example. Instead, we must discover what the common attributes are [56]. We will use schema matching for this purpose [63]. Notice that we are not given just two schemas that need to be matched, rather we have a set of tables all of which could potentially share attributes with some or many of the other tables. Hence, we will use *holistic schema matching* [64]. 2) We cannot assume that integrated datasets are complete (that is, they may actually contain null values or partial facts). 3) Prior work on Full Disjunction has been done on relatively small relations (with only 1000 or so tuples per relation [17]) or assumes the common attributes form graphs with specific acyclic structures [65]. To the best of our knowledge, the only work on using FD on larger datasets requires that all joins be done on attributes having key-to-foreign-key relationships [59], a strict requirement that makes the technique only applicable within well-designed enterprise scenarios, not the possibly messy tables retrieved from data lakes commonly used in data science.

We assume data scientists use table discovery algorithms to identify a set of tables that they wish to integrate. Regardless of the search technique, we wish to find the best way to integrate the tables. Specifically, we propose a table integration technique ALITE (Align and Integrate) that first applies schema matching to identify common columns in a set of tables to be integrated. Matched columns are given the same integration ID. We then apply a natural FD over the tables using integration IDs as attribute names.

Contributions. (1) To the best of our knowledge, we introduce the problem of integrating data lake tables obtained using table discovery algorithms. (2) We propose a new holistic schema matching algorithm for sets of tables that outperforms the state-of-the-art matchers on the real data lake tables in our integration benchmarks. (3) We compare the FD used as the integration semantics in ALITE to several other semantics and show the difference and superiority of FD. Empirically, we show FD's superiority for a downstreaming task of entity resolution [44]. (4) We propose a novel algorithm to compute the FD by using complementation and subsumption operators in a novel way. We show that the use of these operators permits optimizations that make the computation faster than the state-of-the-art techniques, in practice. Specifically, ALITE scales better than the state-of-the-art FD algorithms on data lake tables, which are typically large and may have complex join graphs. (5) We introduce and share several open data integration benchmarks.

2 PRELIMINARIES

We now provide the building blocks for integrating tables in a data lake setting, namely, specifying the notation and the basic integration operators, after which we formally define the problem.

Table 1: Symbols used in this paper and their definitions

Symbol	Definition	Symbol	Definition
$\mathcal{T}$ (n=$\|\mathcal{T}\|$)	Set of Tables	r	Set of tuples
T (T_i)	Table (the i_{th} table in $\mathcal{T}$)	$t[A]$	Value of t on the column A
A ($T.A$)	Column (Column A in table T)	$\mathcal{S}$ (s=$\|\mathcal{S}\|$)	Set of all input tuples
m_i	Arity of Table T_i	$\mathcal{F}$ (f=$\|\mathcal{F}\|$)	Set of output tuples
$\mathcal{A}(T)$	Schema (set of columns) of T	±	Null denoting a missing value
t	Tuple	⊥	Null produced by an operator

Notation. Table 1 summarizes the used notation. For any operator, let $\mathcal{S}$ (and its size s) and $\mathcal{F}$ (and its size f) denote the collective set of all input and output tuples, respectively. We use two types of nulls: ± denotes a *missing null* i.e., missing value from an incomplete input relation to be integrated and ⊥ denotes a *produced null*, a null value that is introduced by an operator during integration.

TID	Stadium	Location	Team	Opened	Coach	Capacity
t_1	NRG Stadium	Texas	Houston Texans	⊥	⊥	⊥
t_2	AT&T Stadium	Texas	Dallas Cowboys	⊥	⊥	⊥
t_3	Paul Brown	Ohio	±	⊥	⊥	⊥
t_4	Sofi Stadium	California	Angeles Chargers	⊥	⊥	⊥
t_5	Soldier Field	Chicago	⊥	1924	⊥	⊥
t_6	Ford Field	Michigan	⊥	2002	⊥	⊥
t_7	⊥	Texas	Houston Texans	⊥	Lovie Smith	⊥
t_8	⊥	Wisconsin	Green Bay Packers	⊥	Matt LaFleur	⊥
t_9	⊥	Michigan	Detroit Lions	⊥	Dan Campbell	⊥
t_{10}	NRG Stadium	Texas	⊥	⊥	⊥	±
t_{11}	Ford Field	Michigan	⊥	⊥	⊥	65k
t_{12}	Lambeau Field	Wisconsin	Green Bay Packers	⊥	⊥	⊥
t_{13}	±	Ohio	Cleveland	⊥	⊥	⊥
t_{14}	Sofi Stadium	California	±	⊥	⊥	⊥

(a) T1 ⊎ T2 ⊎ T3 ⊎ T4 ⊎ T5

OID	TIDs	Stadium	Location	Team	Opened	Coach	Capacity
f_1	$\{t_1, t_7, t_{10}\}$	NRG Stadium	Texas	Houston Texans	⊥	Lovie Smith	±
f_2	$\{t_2\}$	AT&T Stadium	Texas	Dallas Cowboys	⊥	⊥	⊥
f_3	$\{t_3\}$	Paul Brown	Ohio	±	⊥	⊥	⊥
f_4	$\{t_{13}\}$	±	Ohio	Cleveland	⊥	⊥	⊥
f_5	$\{t_4\}$	Sofi Stadium	California	Angeles Chargers	⊥	⊥	⊥
f_6	$\{t_5\}$	Soldier Field	Chicago	⊥	1924	⊥	⊥
f_7	$\{t_6, t_9, t_{11}\}$	Ford Field	Michigan	Detroit Lions	2002	Dan Campbell	65k
f_8	$\{t_8, t_{12}\}$	Lambeau Field	Wisconsin	Green Bay Packers	⊥	Matt LaFleur	⊥
f_9	$\{t_3, t_{13}\}$	Paul Brown	Ohio	Cleveland	⊥	⊥	⊥
f_{10}	t_{14}	Sofi Stadium	California	±	⊥	⊥	⊥

- $FD(T_1, T_2, T_3, T_4, T_5) = \{f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8\}$
- $FD_{tuple\text{-}set}(T_1, T_2, T_3, T_4, T_5) = FD(T_1, T_2, T_3, T_4, T_5) \cup \{f_{10}\}$
- $T_1 \boxplus T_2 \boxplus T_3 \boxplus T_4 \boxplus T_5 = FD(T_1, T_2, T_3, T_4, T_5) - \{f_3, f_4\} \cup \{f_9, f_{10}\}$

(b) Output tuples generated using different operators

Figure 2: Result of integrating the tables in Fig. 1 using different techniques. The table in (a) is the result of outer unioning the five tables. The table in (b) is the union of tuples obtained using FD (first eight tuples in mustard), a variant called tuple-set FD (which is the FD plus f_{10}), and Complement Union (⊞). A unique Output ID (OID) is provided for each output tuple for clarity.

Example 4. *Consider Table T_1 of Fig. 1. The schema of T_1 is $\mathcal{A}(T_1) = \{Stadium, Location, Team\}$. Tuple t_3 = {Paul Brown, Ohio, ±} has attribute value $t_3[Stadium]$ = Paul Brown and a missing null on the Team column, i.e., $t_3[Team] = \pm$.*

2.1 Finding Common Columns

Our running example is unrealistic as common (or in relational terms join-consistent) columns from different tables have the same name and columns that are not common have different names. This is not the case in most realistic examples. Hence, we begin by using schema matching to assign *integration ids* to columns such that two matched columns will have the same id and two columns that are not matched will have different ids. We will ensure no two columns in the same table share an integration id. Accordingly, we will set $\mathcal{A}(T)$ to be the set of integration ids of T's columns (Section 4).

2.2 Integration Operators

We assume that the reader is familiar with the elementary relational algebra operators like union (∪), join (⋈) and outer join(⟗) [66] based on which, we now introduce some (less well known) operators that we use as components of an integration solution.

Outer Union (⊎) is an extension to the union operator. It unions tables even if they do not have the same schema [16]. The outer union between T_1 and T_2 is denoted by $T_1 \uplus T_2$. For each $A \in \mathcal{A}(T_1) - \mathcal{A}(T_2)$, we pad T_2 with a new column A containing nulls (specifically ⊥). Similarly, for each $A \in \mathcal{A}(T_2) - \mathcal{A}(T_1)$, we pad T_1 with a new column A containing nulls. We then union the padded relations.

Example 5. *The outer union of the tables in Fig. 1 is shown in Fig. 2(a). Here, the input size ($|\mathcal{S}| = 14$) is the same as the output size ($|\mathcal{F}| = 14$) but the output may be smaller if there are duplicates.*

Subsumption (β). Given two different tuples t_1 and t_2 having the same schema, the tuple t_1 (subsuming tuple) *subsumes* t_2 (subsumed tuple), denoted by $t_1 \sqsupset t_2$, if all the non-null values of t_2 are equal to that of t_1 on the respective columns and t_1 has fewer null values (either missing or produced) than t_2 [7, 32]. We denote the subsumption operation using β where $\beta(r)$ contains all tuples of r that are not subsumed by another tuple in r. Applying subsumption to the outer union result is called the *minimum union* (⨁) [32]. An example of minimal union (⨁) of tables in Fig. 1 is the set $\{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{11}, t_{12}, t_{13}\}$ in Fig. 2. This is because the tuple t_{10} is subsumed by t_1 and t_{14} is subsumed by t_4. Here, the size of the input ($|\mathcal{S}| = 14$) is larger than the output ($|\mathcal{F}| = 12$).

Complementation (κ). Two different tuples t_1 and t_2 having the same schema *complement* each other if: 1) there is at least one column A on which they have equal and non-null values; 2) for every column A where both tuples are non-null, the tuples must have the same value on A; 3) there is at least one column A on which t_1 is non-null and t_2 is null (missing or produced); and 4) there is at least one column A on which t_2 is non-null and t_1 is null (missing or produced) [7, 9]. The complementation of t_1 and t_2 is a tuple t_3 where for any column A, $t_3[A] = t_1[A]$ if either $t_1[A]$ is non-null or both $t_1[A]$ and $t_2[A]$ are non-null (hence, equal). Otherwise, if $t_2[A]$ is non-null $t_3[A] = t_2[A]$. For the case where both values are null, if $t_1[A] = t_2[A] = \perp$ then $t_3[A] = \perp$ otherwise (at least one of the nulls is missing) $t_3[A] = \pm$. The complementation operator (κ) replaces all complemented pairs of tuples with their complementation. Note that a tuple that results from complementation could be complemented by other tuples so the complementation operator is the iterative result of applying complementation to a relation until it contains no further complementing tuples. Applying complementation over a set of outer unioned tuples, is known as *complement union* (⊞).

Example 6. *In Table(a) of Fig. 2, tuples t_3 and t_{13} complement each other. Their complementation is denoted as f_9 in Fig. 2(b), i.e., $\kappa(t_3, t_{13}) = f_9$. So complementation can* overcombine *tuples that do not agree on all their common attributes. In this example, T_5 asserts that* `Cleveland` *is a team in* `Ohio` *with an unknown stadium while T_1 asserts that* `Paul Brown` *is a stadium in* `Ohio`*. But we do not definitively know that* `Paul Brown` *is the stadium of* `Cleveland`*. The complement union of the tables in Fig. 1, i.e., $T_1 \boxplus T_2 \boxplus \cdots \boxplus T_5$ is the set of tuples in Table (b) of Fig. 2 excluding tuples $\{f_3, f_4\}$. Note that complementation union may not remove all subsumable tuples (e.g. f_{10}, which can be subsumed by f_5, is not complemented by f_5 since the fourth condition of complementation is not met).*

2.3 Full Disjunction

The operators of the previous section offer possible semantics for integrating tables. In 1994, Galindo-Legario proposed a different semantics called Full Disjunction (FD) [32]. His proposal is essentially



Magneto: Combining Small and Large Language Models for Schema Matching

Yurong Liu*
New York University
yurong.liu@nyu.edu

Eduardo H. M. Pena*†
Federal University of Technology Paraná
eduardopena@utfpr.edu.br

Aécio Santos
New York University
aecio.santos@nyu.edu

Eden Wu
New York University
eden.wu@nyu.edu

Juliana Freire
New York University
juliana.freire@nyu.edu

ABSTRACT

Recent advances in language models (LMs) open new opportunities for schema matching (SM). Recent approaches have shown their potential and key limitations: while small LMs (SLMs) require costly, difficult-to-obtain training data, large LMs (LLMs) demand significant computational resources and face context window constraints. We present `Magneto`, a cost-effective and accurate solution for SM that combines the advantages of SLMs and LLMs to address their limitations. By structuring the SM pipeline in two phases, retrieval and reranking, `Magneto` can use computationally efficient SLM-based strategies to derive candidate matches which can then be reranked by LLMs, thus making it possible to reduce runtime while improving matching accuracy. We propose (1) a self-supervised approach to fine-tune SLMs which uses LLMs to generate syntactically diverse training data, and (2) prompting strategies that are effective for reranking. We also introduce a new benchmark, developed in collaboration with domain experts, which includes real biomedical datasets and presents new challenges for SM methods. Through a detailed experimental evaluation, using both our new and existing benchmarks, we show that `Magneto` is scalable and attains high accuracy for datasets from different domains.

PVLDB Reference Format:
Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. `Magneto`: Combining Small and Large Language Models for Schema Matching. PVLDB, 18(8): 2681 - 2694, 2025.
doi:10.14778/3742728.3742757

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/VIDA-NYU/magneto-matcher.

*Equal contribution.
†Work done as a visiting researcher at New York University.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 8 ISSN 2150-8097.
doi:10.14778/3742728.3742757

1 INTRODUCTION

The rapid increase in the volume of structured data– from data published in scientific articles [63, 72] and repositories [29, 60, 66, 80] to open government portals [19, 64] – creates new opportunities to answer important questions through analytics and predictive modeling. But often, data from multiple sources must be integrated to answer these questions. Consider the following example.

Example 1.1. In the context of the National Cancer Institute's Clinical Proteomic Tumor Analysis Consortium (CPTAC) [15], Li et al. [52] carried out a comprehensive proteogenomic analysis of cancer tumors. They collected data from ten studies (published as supplementary material in research papers) that cover multiple patient cohorts and cancer types. To facilitate their analysis, they mapped each dataset into the GDC standard, a data model set by the National Cancer Institute's Genomic Data Commons (GDC) for cancer genomic data [38, 60]. Even though the studies had been carried out by members of the CPTAC effort, the datasets containing patient case and sample data used different representations for variable names and values. Integrating these data required a substantial effort to match variables from each dataset source schemata to the target GDC format, which encompasses over 700 attributes. □

Even though the schema matching problem has been extensively studied [8, 22, 48, 58], matching still requires a time-consuming, manual curation process for complex tasks, which like the one described above, involves ambiguity and heterogeneity in the representation of attributes and values (see Table 1). Schema matching approaches that rely on attribute names, data types and values for similarity assessments are likely to fail for such matches. As a point of reference, we assessed the effectiveness of state-of-the-art strategies for the biomedical datasets from Example 1.1. As shown in Figure 1, they perform poorly: the best technique achieves at most 0.45 of mean reciprocal rank (MRR) and incurs high computational costs. We discuss these results in detail in Section 6.

Determining correspondences between columns may require knowledge beyond the schema and contents of a table. Table 1 shows some possible matches for the proteogenomic analysis. Sometimes, selecting the correct match is difficult even for subject matter experts. For example, for the attribute `patient_age` from one of the datasets, there are at least three plausible matches in GDC: `age_at_diagnosis`, `days_to_birth`, and `age_at_index`. Without additional context, it is difficult to determine the correct match.

Table 1: Examples of candidate matches between source tables and the GDC that highlight schema heterogeneity.

Source Column and Values	Target Candidate Columns
Histologic_Grade_FIGO: FIGO grade 1, FIGO grade 2, FIGO grade 3	**tumor_grade**: G1, G2, G3 **who_nte_grade**: G1, G2, G3 **adverse_event_grade**: Grade 1, Grade 2, Grade 3
Path_Stage_Primary_Tumor-pT: pT1a (FIGO IA), pT2 (FIGO II), pT3b (FIGO IIIB)	**uicc_pathologic_t**: T1a, T2, T3b **ajcc_pathologic_t**: T1a, T2, T3b
Age: 65, 72, 49	**age_at_diagnosis**: n/a **days_to_birth**: n/a **age_at_index**: n/a

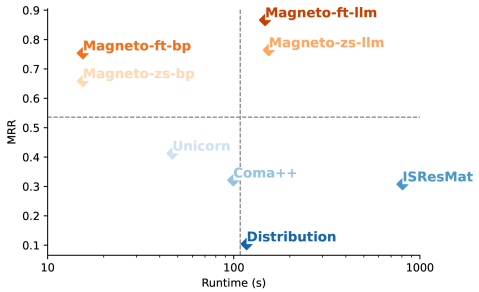


Figure 1: Trade-off between runtime and accuracy (using MRR) for different schema matchers in the task of Example 1.1. `Magneto` variants (orange) outperform traditional methods (blue) in MRR and accuracy-runtime trade-off.

Schema Matching and Language Models. Renewed interest in data integration has emerged due to the capabilities of language models [34, 59]. For schema matching, promising approaches have been proposed [10, 11, 27, 79]. A key challenge in schema matching is estimating the similarity between two columns. Pre-trained Language Models (PLMs), referred to as SLMs in this paper to distinguish them from large language models (LLMs), create column representations (or embeddings) enriched with semantic information. The similarity between two embeddings serve as a proxy for column-matching scores [10, 11]. SLMs have also been fine-tuned for schema matching [27] and general matching tasks [79].

LLMs, in contrast, are trained using large, generic data corpora and thus contain knowledge that can assist in obtaining additional semantics necessary to identify matches. Prompting strategies combined with fine-tuned models have been shown effective to improve table understanding and help in integration tasks [50].

While prior work has shown the usefulness of SLMs and LLMs for schema matching, they also present significant practical challenges:

- *Challenge 1*: Fine-tuning SLMs can lead to significant performance improvements, but this requires the availability of manually curated training data, which can be expensive to create.
- *Challenge 2*: LLMs usually do not need fine-tuning, but face constraints due to fixed context windows, requiring truncation of large prompts and potential loss crucial information. While some models offer larger windows, cost scales with input and output size. Furthermore, accuracy can decline with long prompts and API calls incur high latency, especially for large inputs [51].

Our Approach. We introduce `Magneto`, a framework that combines SLMs and LLMs to derive cost-effective and general schema matching solutions. As illustrated in Figure 2, `Magneto` is structured in two phases: *candidate retrieval* selects a subset of the possible matches; and reranker, which ranks the candidates to make it easier for users to examine and select matches. To address *Challenge 1*, `Magneto` leverages LLMs to automate SLMs fine-tuning. Instead of relying on manually created training data and structure-based augmentation (e.g., row shuffling and sampling [27, 31]), we use LLMs to derive data. Using LLMs, we can add syntactic diversity and capture different representations for column names and their values. `Magneto` addresses *Challenge 2* by using cheaper SLM-based methods for finding candidates thereby reducing the number of matches that need to be checked by more costly LLM-based rerankers. To ensure that all necessary details are included in the prompt while staying within the context window limit, we need to create representative samples of the columns in candidate matches to be ranked by the LLM. In addition, since LLMs are designed for textual data, we must serialize the column content. Selecting the right serialization strategy is still an open research problem that has attracted substantial attention [23, 32]. We explore different alternatives for both sampling and serialization.

Our goal with `Magneto` is to achieve high accuracy for schema matching at a low cost. Figure 1 illustrates the trade-offs between runtime and accuracy for existing schema matching algorithms and different variants of `Magneto`. Note that traditional methods tend to cluster at relatively high accuracy with long runtimes or relatively low runtimes with reduced accuracy. In contrast, `Magneto` variants strike a balance across both dimensions and have the advantage of requiring no training data.

We conduct an extensive experimental study of `Magneto` on various datasets from two benchmarks. The first is a new benchmark, that we developed in collaboration with domain experts, which includes real biomedical datasets. Our experiments show that many existing schema matching solutions struggle with this benchmark, particularly given the complexity and syntactic variability present in its datasets. In contrast, `Magneto` demonstrates superior accuracy and generally faster runtimes. We also evaluate `Magneto` on the Valentine benchmark [48] and observe that it also performs well for the Valentine datasets which cover a variety of domains.

Contributions. Our main contributions are summarized as follows:

- We introduce the `Magneto` framework that effectively combines SLMs and LLMs and allows the creation of schema matching strategies that attain a balance between accuracy and runtime for diverse data and tasks (Section 2).
- We propose an LLM-powered method to generate training data for fine-tuning SLMs for schema matching tasks and a contrastive learning pipeline using triplet loss and online triplet mining to enhance column embedding distinction (Section 3).
- Unlike previous approaches that use LLMs for schema matching, we show that reranking candidate matches with LLMs, derived from less costly methods, achieves high accuracy at a lower cost. (Section 4).
- In collaboration with biomedical researchers, we created a new benchmark that represents a real schema matching effort [52],

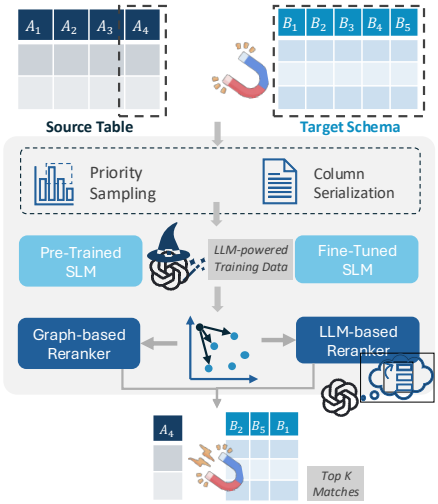


Figure 2: Magneto takes source and target tables and identifies matches in two phases: an SLM retrieves and ranks candidates, then an LLM assesses and reranks them. This enables efficient matching and cross-domain generalization, with customizable strategies based on method combinations.

encompassing characteristics not present in existing benchmarks and introducing new challenges for SM methods (Section 5).

- We perform an extensive experimental evaluation of different strategies derived using our framework, comparing them against SM methods over datasets that cover a wide range of domains. We also carry out ablation studies to assess the effectiveness of different choices for column sampling and serialization (Section 6).

2 MOTIVATION AND SOLUTION OVERVIEW

Biomedical Data Integration: Opportunities and Challenges. Due to substantial investments made in infrastructure to share biomedical data [62], many datasets are now stored in repositories with related research articles [60, 66]. Pooling these data can advance research across various diseases and populations, but current technologies are limited in their ability to integrate these data. Our research was motivated by this challenge as part of a collaboration with biomedical researchers in the context of the ARPA-H Biomedical Data Fabric (BDF) program [2], which aims to develop usable data integration methods and tools.

Our interviews with biomedical experts revealed that current data integration practices rely heavily on manual processes and dataset-specific scripts for schema matching [12, 52, 57]. As illustrated in Example 1.1, this approach is both error-prone and difficult to reproduce, creating significant barriers to biomedical research.

Based on these findings, we identified two critical requirements that guided the design of Magneto. First, researchers need approaches that generalize across the diverse landscape of biomedical data—spanning different data types (genomics, proteomics, electronic health records), disease categories (various cancers, autoimmune and rare diseases), and data sources (publications, hospitals, data commons). Second, given the complexity of biomedical schemas, these approaches must facilitate curation, as even subject matter experts often struggle to determine correct matches.

2.1 Definitions and Evaluation Metrics

Before describing our approach, we introduce the notation, schema matching definition, and evaluation metrics used in this paper.

Definition 2.1. (Schema Matching) Let $\mathbf{S}(A_1, \ldots, A_n)$ be a source table and $\mathbf{T}(B_1, \ldots, B_m)$ be a target table, where $A_i \in \mathbf{S}$ and $B_j \in \mathbf{T}$ are columns that define the schemata. Each column $A \in \mathbf{S} \cup \mathbf{T}$ has an associated *domain*, denoted $\mathcal{D}(A)$, representing the set of possible values that the column can take; note that $\mathcal{D}(A)$ may be empty. Schema matching focuses on aligning the table schemata by establishing correspondences between columns representing the same real-world concept or entity. A matching algorithm (or matcher) aims to identify pairs (A_i, B_j) that represent the same (or semantically equivalent) column based on various factors, such as their domains and names. Thus, a matcher $\mathcal{M}$ can be seen as a function that generates a schema mapping $M \subseteq \mathbf{S} \times \mathbf{T}$, where each element $(A, B) \in M$ represents a correspondence between a source column A and a target column B, where $\mathcal{D}(A) \approx \mathcal{D}(B)$, meaning that the domains of columns A and B are related or overlap. □

Matching algorithms often associate a score with each match they derive. These scores are used to generate ranked lists containing the derived matches, which help users explore matches by prioritizing the highest-scoring candidates. These lists provide a global ranking of best matches (A_i, B_j) among all pairs of possible matches of a given pair of tables $\mathbf{S}$ and $\mathbf{T}$ or a per-column ranking of the best matches B_j for a given source attribute $A_i \in \mathbf{S}$. Thus, a common approach to evaluate schema matching methods is to assess their ability to produce high-quality ranked lists of matches [27, 48]. We use two evaluation metrics: Mean Reciprocal Rank (MRR) and Recall at Ground-Truth Size (Recall@GT), as detailed next.

Definition 2.2. (Mean Reciprocal Rank) Let $matches[A]$ denote the ranked list of matches produced by a schema-matching algorithm for the source column $A \in \mathbf{S}$, and r_A be the position of the first correct target column B within the ordered list $matches[A]$. The reciprocal rank (RR) of an individual column A is the multiplicative inverse of the rank, i.e., $\frac{1}{r_A}$. The *mean reciprocal rank* (MRR) for a table $\mathbf{S}$ is the average RR over the subset of columns $\mathbf{S}'$ that contain a correct match in the ground truth:

$$\text{MRR} = \frac{1}{|\mathbf{S}'|} \sum_{A \in \mathbf{S}'} \frac{1}{r_A}. \tag{1}$$

Intuitively, MRR measures how long it takes to find the first relevant match when examining the ranked list of matches. □

MRR is a standard evaluation metric for ranked lists in search engines and question-answering systems [71, 83]. For schema matching, high-quality results correspond to ranking the most relevant match for each source column as highly as possible. A high MRR score therefore indicates that users can more easily identify correct matches when evaluating candidate matches for a given column.

We also use the recall at ground truth size (Recall@GT), a standard measure used in recent schema matching literature [48]. Unlike MRR, which evaluates rankings per source column, Recall@GT operates on a global ranking that merges all candidate matches across columns into a single list.

Definition 2.3. (Recall@GT) Let *matches* denote the global ranked list containing all matches produced by a matcher that considers all

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00123

Schema Matching using Pre-Trained Language Models

Yunjia Zhang
University of Wisconsin-Madison
yunjia@cs.wisc.edu

Avrilia Floratou
Microsoft
avflor@microsoft.com

Joyce Cahoon
Microsoft
jcahoon@microsoft.com

Subru Krishnan
Microsoft
subru@microsoft.com

Andreas C. Müller
Microsoft
amueller@microsoft.com

Dalitso Banda*
Sentry
dalitso.banda@sentry.io

Fotis Psallidas
Microsoft
fopsalli@microsoft.com

Jignesh M. Patel
University of Wisconsin-Madison
jignesh@cs.wisc.edu

***Abstract*—Schema matching over relational data has been studied for more than two decades. However, the state-of-the-art methods do not address key modern-day challenges encountered in real customer scenarios, namely: 1) no access to the source (customer) data due to privacy constraints, 2) target schema with a much larger number of entities and attributes compared to the source schema, and 3) different but semantically equivalent entity and attribute names in the source and target schemata. In this paper, we address these shortcomings. Using real-world customer schemata, we demonstrate that existing linguistic matching approaches have low accuracy. Next, we propose the Learned Schema Mapper (LSM), a novel linguistic schema matching system that leverages the natural language understanding capabilities of pre-trained language models to improve the overall accuracy. Combining this with active learning and a smart attribute selection strategy that selects the most informative attributes for users to label, LSM can significantly reduce the overall human labeling cost. Experimental results demonstrate that users can correctly match their full schema while saving as much as 81% of the labeling cost compared to manual labeling.**

I. Introduction

More and more organizations in various industries (retail, healthcare, manufacturing, etc.) are adopting a data-driven approach to discover non-trivial and strategic insights. At the same time, the amount of information collected has never been greater [1], highlighting the need for modern data analytics tools that can keep up with the volume, variety, and velocity of data associated with different industries. To that end, multiple companies are offering industry-specific analytics solutions [2]–[4] tailored to meet customers' domain-specific needs. Azure Industry solutions [2], for example, offers a variety of services that aim to optimize industry-specific processes, provide enhanced collaboration capabilities and reduce the time to actionable insights. Enabling these enhanced experiences often requires a substantial understanding of customer data. To tackle this problem, the service operator usually maps the customer's data to a known industry-specific schema (ISS) whose semantics are clearly defined, and then builds tools on top of it to streamline various processes.

*Work done while at Microsoft.

In this paper, we study the schema matching problem in the context of known ISSs and customer data in the form of a relational schema. While the problem has been studied for over two decades, we find that existing solutions are not adequate in our environment.

First, information about customer data is often limited due to privacy considerations. In particular, we find that customers are reluctant to grant access to individual data records [5], [6]. As a result, the service operator cannot leverage customer data records when performing schema matching. The same considerations do not constrain their access to the customer schemata. Indeed, since the outcome of schema matching reveals what entities and attributes in the known ISS are present in the customer data, further obfuscating the customer schema from the service operator is not valuable [1]. As a result, customers are generally willing to provide schema information to the operator to facilitate schema matching.

Second, ISSs are typically large (in terms of entities and attributes) as they try to capture a wide variety of concepts associated with a given industry. The customer schema, however, does not necessarily encapsulate all these concepts and is often much smaller than the target ISS. This discrepancy can complicate the schema matching process as it results in a large number of available (and potentially irrelevant) candidate matches for each attribute of the customer schema.

Finally, the entity and attribute names of the customer schema are often hard to understand due to abbreviations and/or customer-specific terminology, further complicating the automation of schema matching for the service operator.

The externally-imposed customer constraints along with the challenges associated with real customer schemata significantly limit the accuracy of existing schema matching approaches (see Section III). In this work, we propose the Learned Schema Matcher (LSM), a novel human-in-the-loop linguistic schema matching approach. LSM does not require access to the individual data records (data-free schema matching) but only to the source schema, respecting the customer

[1]There are other scenarios where both the source and target schemata need to be obfuscated from the service operator [7]. Extending our approach to meet these more stringent privacy requirements is left for future work.

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00123

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

requirements. At the core of our approach lies a fine-tuned language model that is able to scale to large ISSs and allows us to better handle noise in the source schema. Further, by combining this model with active learning and a smart interaction strategy to obtain targeted feedback from the user, we can fully map a source schema to an ISS while incurring significantly less human labeling cost than prior work.

Our contributions are the following:

- We study the state-of-the-art linguistic schema matching approaches on real-world schemata from Microsoft customers and demonstrate that they have low accuracy.
- We develop LSM, a novel matching approach that relies on a fine-tuned pre-trained language model to tackle data-free schema matching. To the best of our knowledge, this is the first work demonstrating that leveraging the natural language capabilities of pre-trained language models can improve the accuracy of data-free schema matching. Previous methods that are either not data-free [8]–[10] or do not leverage natural language understanding [11]–[15].
- The externally-imposed customer constraints render near-100% accuracy virtually impossible without humans-in-the-loop. We thus propose to combine our schema matching model with an active learning strategy. Our method employs a novel attribute selection strategy to obtain targeted feedback from the user with the goal of reducing the human labeling cost.
- We evaluate LSM on real customer schemata and publicly available datasets. We show that with LSM, users can correctly match their full schema to ISS with a 81% reduction in labeling cost compared to manual labeling.

The remainder of the paper is organized as follows: Section II contains the problem statement. Section III presents an evaluation of the state-of-the-art linguistic matching approaches on real customer schemata. Section IV describes the design of our proposed LSM method. Section V contains our experimental evaluation, and Section VI discusses our experiences in designing LSM. Finally, Section VII presents the related work, and our conclusions are stated in Section VIII.

II. Problem Statement

We now define the problem of schema matching in our environment. Given a source (customer) schema and a known target (ISS) schema that both follow the E/R model [16], our goal is to map each attribute of the source schema to an attribute at the target schema. Figure 1 shows an example of an ISS and a customer schema and their corresponding match.

More formally, we are given a source schema S_s consisting of a set of entities E_s, a set of attributes A_s, and a set of PK/FK relationships R_s. Similarly, we also have a known target schema S_t that consists of a set of entities E_t, a set of attributes A_t, and a set of PK/FK relationships R_t. We assume that each attribute in A_s (A_t) belongs to a single entity in E_s (E_t).

Each entity e ($e \in E_s \cup E_t$) has a name $e.name$, a primary key $e.pk$, and a set of foreign keys $e.fks$. Each attribute a ($a \in A_s \cup A_t$) has a name $a.name$, a data type $a.dtype$, and optionally a natural language description $a.desc$ associated with it.

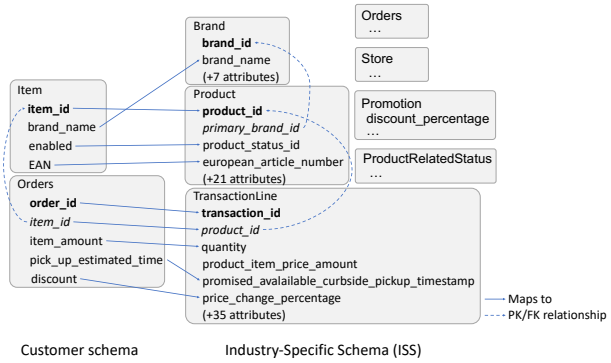


Fig. 1: An example customer schema and corresponding ISS.

We define an attribute correspondence as $r_{ij} = (a_i, a_j)$ where a_i is an attribute in the source schema S_s and a_j is an attribute at the target schema S_t. As discussed in [17], intuitively, an attribute correspondence specifies a relationship between a_i and a_j. The correspondence may specify that the two attributes are equal to each other or that there may be a transformation function involved (e.g., from dollars to euros). In this work, we focus on correspondences that denote equality and leave more complex transformations as part of future work.

Definition 1. *An entity match $\mathcal{M}$ is a triple (e_s, e_t, m), where e_s is an entity in E_s, e_t is an entity in E_t, m is a set of attribute correspondences between e_s and e_t, and each source and target attribute occurs in at most one correspondence in m.*

Based on the above definition, we now define the schema matching problem addressed in this paper.

Definition 2. *The result $\overline{\mathcal{M}}$ of the schema matching process is a set of entity matches between entities in E_s and in E_s, where each attribute in S_s and S_t appears in at most one entity match.*

As we discuss in Section IV, our approach generates $\overline{\mathcal{M}}$ by utilizing all the available information in the source and target schemata (i.e., entities, attributes, and PK/FK relationships).

III. Motivation

Given that in our setting, we do not have access to the customer data, we evaluated the applicability of the state-of-the-art schema matching approaches that rely solely on schema-level information. In this section, we demonstrate that prior approaches have poor accuracy on our real customer schemata. We then identify a set of challenges that motivate the design of the Learned Schema Matcher.

State-of-the-art Methods. We choose four representative schema-based matching methods: CUPID [11], COMA [18],

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

	# Entities	# Attr.	# Unique Attr. Names	# PK/FK	Desc.
Customer A	3	29	28	2	Y
Customer B	8	53	45	7	N
Customer C	3	84	82	2	N
Customer D	7	136	125	7	N
Customer E	25	530	475	24	Y

TABLE I: Statistics on the customers' (source) schemata.

		# Entities	# Attributes	# PK/FK
RDB-star	Source	13	65	12
	Target	5	34	4
IPFQR	Source	1	51	0
	Target	1	67	0
MovieLens-IMDB	Source	6	19	5
	Target	7	39	6

TABLE II: Statistics on publicly available schemata.

S-MATCH [12], and Similarity Flooding [15], which are all heuristic-based. To evaluate the effectiveness of machine learning-based approaches, we also experimented with the LSD [13] and MLM [14] methods. Note that these methods require access to both the schema and the data records, which is not possible in our environment. We thus had to adapt them to leverage schema-level information only. We implemented all the methods except COMA [18] from scratch according to the descriptions in [11]–[14] respectively. We use the latest community edition of COMA 3.0 [2]. Note that COMA and CUPID can optionally accept user feedback during the matching process. In this section, we present results without user feedback, but we do use the interactive mode in our experiments in Section V where we present a comparison with our approach. A summary of the methods is presented below:

- ***CUPID:*** CUPID tackles the schema matching problem for both XML and relational schemata by combining both linguistic (through a synonym dictionary) and structural information and computing the weighted sum of the associated similarities. In our implementation, we use the pre-trained word embedding from FastText [19] as the synonym dictionary and generate the similarity score using cosine similarity. For each customer schema, we search the best-performing weights for the weighted sum and report only the best results.
- ***COMA:*** COMA uses a set of matchers to compute the linguistic similarity between two attributes. The matchers cover a broad spectrum of similarity metrics such as affix, n-gram, Soundex, edit distance, etc. To combine the similarities, COMA can choose from various aggregation functions such as min, max, average, etc.
- ***S-MATCH:*** S-MATCH (SM) solves the semantic matching problem on two trees. It uses WordNet [20] to understand the meaning of the nodes in the trees and identify synonyms. It covers multiple semantic relations such as equivalence, mismatch, overlapping, etc. We use S-MATCH to tackle the schema matching problem as defined in Definition 2 and thus consider only attribute equivalence.

[2]https://sourceforge.net/projects/coma-ce/

	CUPID	COMA	SM	SF	LSD	MLM
RDB-Star	0.96	**1.00**	**1.00**	0.70	0.26	**1.00**
IPFQR	**1.00**	0.98	0.82	**1.00**	0.08	0.98
MovieLens-IMDB	0.64	0.54	**0.72**	0.71	0.00	0.64
Customer A	0.18	**0.21**	0.07	0.11	0.00	0.11
Customer B	**0.14**	0.02	0.06	0.02	0.00	0.08
Customer C	0.08	0.22	0.11	**0.23**	0.00	0.14
Customer D	0.27	**0.31**	0.13	0.14	0.00	0.14
Customer E	0.27	0.17	**0.28**	0.12	0.00	0.14

TABLE III: The top-3 accuracy of six state-of-the-art approaches on our schemata.

- ***Similarity Flooding:*** Similarity flooding (SF) [15] builds a pairwise connectivity graph based on the structure of the source and target schemata. The connectivity graph contains element pairs as nodes. Two nodes (x_1, y_1) and (x_2, y_2) are connected if x_1, x_2 and y_1, y_2 are neighbors in the original schema graphs respectively. SF initializes the similarity scores of the element pairs and propagates the scores for each element pair following the edges in the connectivity graph. In our implementation, we use embedding similarities as the initial scores.
- ***LSD:*** The LSD method is one of the first to leverage machine learning for schema matching. LSD learns to match between the source and target schemata based on a set of provided examples. LSD has four individual learners: 1) the `whirl` learner, which classifies the attributes based on nearest neighbors of TF-IDF encodings, 2) a naive Bayesian learner that classifies the descriptions, 3) a name matcher based on name similarities, and 4) a county-name recognizer that matches the names if the attribute is a county name. We randomly select 50% of the ground truth matches as the training set and measure the matching quality for the rest 50% of the attributes.
- ***MLM:*** MLM is another machine learning-based method. Different from LSD, MLM featurizes the candidate matches using both the schema specifications and the data records. Following the featurization, MLM generates matches with a clustering model (K-means or self-organizing map [14]). To apply MLM in our problem setting, we only use the features from the schema-level specification. Since MLM adopts unsupervised learning, all the attributes in the target (ISS) schema are treated as the training set.

Datasets. We use five real-world schemata from Microsoft customers in retail. The characteristics of these schemata are shown in Table I. As shown in the table, the five schemata are of different sizes: Customer A has the smallest schema with 29 attributes spread across three entities, while Customer E has the largest schema with 25 entities and 530 attributes. Among the five customer schemata, two of them contain attributes with natural language descriptions. The corresponding target ISS is also defined for the retail industry, which encapsulates information about customers, goods, stores, promotions, sales, and other assets that are typically tracked. The retail ISS we consider consists of 92 entities, 1218 attributes, and 184 PK/FK relationships. The ground truth matches are obtained from domain experts.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.